

0924 깃허브 제출용

■ 마감일	@2025/09/24
🔗 프로젝트	BOAZ
■ 업데이트 시간	@2025년 9월 24일 오전 3:02

Yolo

Abstract

- 이전 연구들은 객체 탐지를 위해 분류 모델을 변형해 사용했지만, 우리는 객체 탐지를 회귀 문제로 재구성하여, 이미지 내의 경계 상자와 해당 클래스 확률을 예측
- 단일 신경망이 한 번의 평가만으로 전체 이미지를 분석하고 경계 상자과 클래스 확률을 예측 (One-Stage Detector)
- YOLO는 탐지 파이프라인 전체를 하나의 신경망으로 통합했기 때문에 탐지 성능을 기준으로 End-to-End 최적화 가능
- YOLO는 매우 빠르며 기본 모델은 초당 45 프레임으로 실시간 처리가 가능하며, 더 작은 Fast YOLO는 놀랍게도 155 프레임을 처리하면서도 다른 실시간 탐지기보다 두 배 높은 mAP 성능을 달성
- 최신 탐지 시스템과 비교했을 때, YOLO는 물체 위치를 정밀하게 잡아내는 데 약간 부족할 수 있지만, 배경과 객체를 잘 구분하고 여러 도메인에서 객체 탐지 가능

1. Introduction

- 기존 객체 탐지 시스템들은 보통 분류 모델을 변형해서 탐지 수행
 - 예를 들어, DPM(Deformable Part Models) 같은 시스템은 이미지 전체를 탐색하면서, 각 위치와 크기에서 분류기를 실행. 이런 방식을 슬라이딩 윈도우(sliding window) 방식이라고 함
 - 반면, R-CNN은 후보 영역(Region Proposal)을 찾아내고, 그 후보들에 대해 분류기를 실행
 - 이 과정이 끝나면, 경계 상자(Bounding Box)를 조정하거나 중복 탐지를 제거하고, 물체 간 상호작용을 고려해 점수를 다시 매기는 후처리(Post-

Processing)가 필요

- 문제는 이런 방식들이 과정이 복잡하고 느리며, 각 단계별로 따로 학습해야 한다는 점
- YOLO는 객체 탐지를 이미지 픽셀에서 바로 bounding box 좌표와 class 확률을 예측하는 단일 회귀 문제로 재구성
- 이 시스템을 통해 이미지를 한 번만 보고 어떤 물체들이 어디에 있는지를 예측할 수 있음
 - YOLO는 단일 컨볼루션 신경망(Convolutional Neural Network)을 사용해 여러 bounding box 와 그 class 확률을 동시에 예측
 - 슬라이딩 윈도우나 지역 제안(Region Proposal) 기반 기법은 이미지 일부만 보기 때문에 물체의 문맥적 정보(Context)를 놓치기 쉬움
- YOLO는 탐지 파이프라인이 복잡하지 않아서 매우 빠름
 - 기본 YOLO 모델은 Titan X GPU에서 초당 45프레임(45 fps)을 처리하며, 빠른 버전(Fast YOLO)은 초당 150프레임 이상을 처리
- YOLO는 물체의 일반적인 표현(Generalized Representations)을 학습
 - 새로운 데이터 도메인에서도 좋은 성능을 보임
- 하지만 YOLO는 아주 정확한 위치를 잡아내는 데는 약간 약할 수 있다는 점

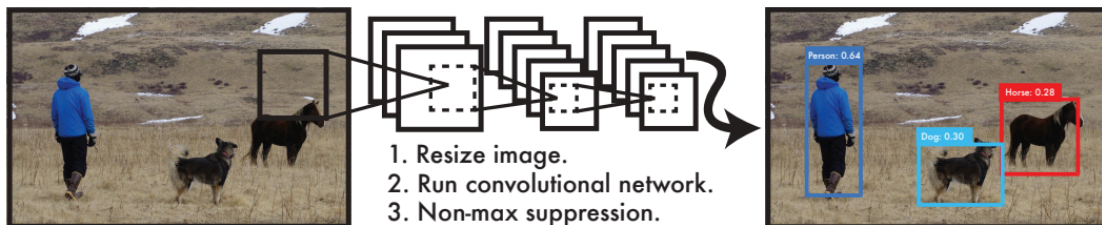


Figure 1: The YOLO Detection System. Processing images with YOLO is simple and straightforward. Our system (1) resizes the input image to 448×448 , (2) runs a single convolutional network on the image, and (3) thresholds the resulting detections by the model's confidence.

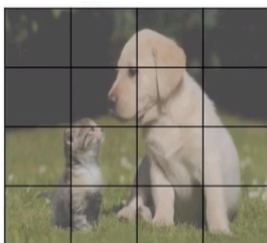
2. Unified Detection

- YOLO는 입력 이미지를 $S \times S$ 크기의 그리드로 나눔

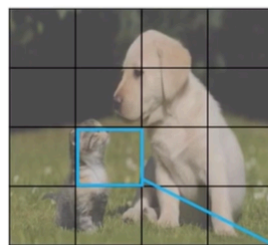
- 예를 들어, PASCAL VOC에서는 $S=7$ 로 설정되어 있으니 이미지가 7×7 로 분할
- 각 그리드 셀은 B개의 Bounding Box와 그 상자들의 Confidence Score를 예측
- bounding box는 5가지 정보를 포함
 - x, y : 상자의 중심 좌표(그리드 셀 내 상대적인 위치/ 0~1)
 - w, h : 상자의 너비와 높이(전체 이미지에 대한 비율로 예측/ 0~1)
 - Confidence
 - $P(\text{Object}) \times \text{IOU}$
pred, truth
 - $P(\text{Object})$: 해당 bounding box 안에 실제로 물체가 존재할 확률
 - IOU : 예측된 bounding box와 실제 bounding box의 겹치는 부분을 기준으로 계산한 일치율
pred, truth
 - 물체가 없다면 Confidence는 0
- 각 그리드 셀은 물체가 포함된 경우, 해당 물체가 특정 클래스일 확률 $\text{Pr}(\text{Class} | \text{Object})$ 도 예측
 - C: 클래스의 수 (예: PASCAL VOC에서는 $C=20$)
 - **중요:** 그리드 셀당 하나의 클래스 확률 집합만 예측하며, 상자 개수(B)와는 무관

Unified Detection

예시) $S = 4, B = 2, C = 20$



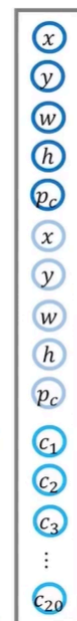
Resized image 를
4x4 grid 로 분할



Grid cell 마다 bbox 2개씩 예측

bbox #1

bbox #2



x
 y
bbox의 중심좌표의 위치
(grid cell 기준)

w
 h
Input image W, H 로 normalize

p_c : $\text{Pr}(\text{Object}) \times \text{IOU}_{\text{pred}}^{\text{truth}}$

cf. $\text{Pr}(\text{Object})$:
물체가 bbox 내에 있으면 1,
없으면 0

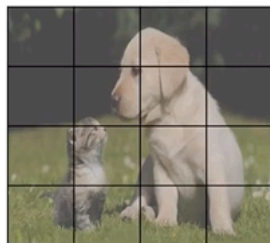
$\text{Pr}(\text{Class}_i | \text{Object})$

: 물체가 bbox 내에 있을 때,
Grid cell에 있는 object가 i번째
class 에 속할 확률

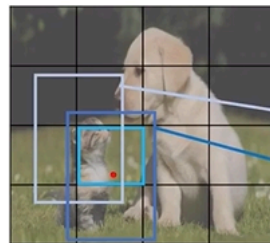
- YOLO의 PASCAL VOC 데이터셋 실험 설정
 - $S=7$: 이미지를 7×7 그리드로 나눔
 - $B=2$: 각 그리드 셀이 2개의 경계 상자를 예측
 - $C=20$: 데이터셋에 20개의 클래스가 존재
 - 최종 출력은 $7 \times 7 \times 30$ 텐서로 나타남
 - $30 = B \times 5 + C = 2 \times 5 + 20$

Unified Detection

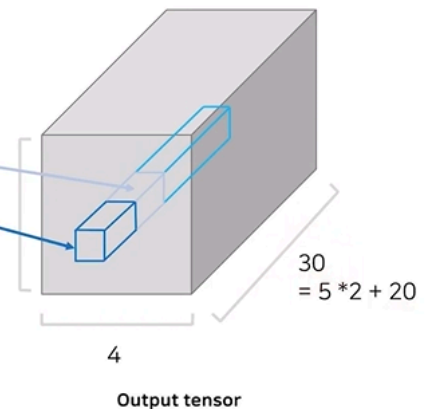
예시) $S = 4, B = 2, C = 20$



Resized image 를
4x4 grid 로 분할



Grid cell 마다 bbox 2개씩 예측



16

2.1 Network Design

- YOLO의 네트워크는 GoogLeNet에서 영감을 받음
- 24개의 Convolution Layer: 이미지의 특징을 추출
- 2개의 FC Layer: 물체의 클래스와 위치를 예측
- GoogLeNet과 달리, Inception 모듈을 사용하지 않음
- 1×1 크기의 축소(reduction) 레이어와 3×3 크기의 컨볼루션 레이어를 사용
- 최종적으로, 네트워크의 출력은 $7 \times 7 \times 30$ 크기의 텐서
- YOLO의 빠른 버전인 Fast YOLO는 더 작은 네트워크 구조를 사용

- 9개의 Convolution Layer: 일반 YOLO보다 적은 레이어
- 각 레이어에 적은 필터: 계산량 감소
- YOLO와 Fast YOLO의 학습 및 테스트 파라미터는 동일

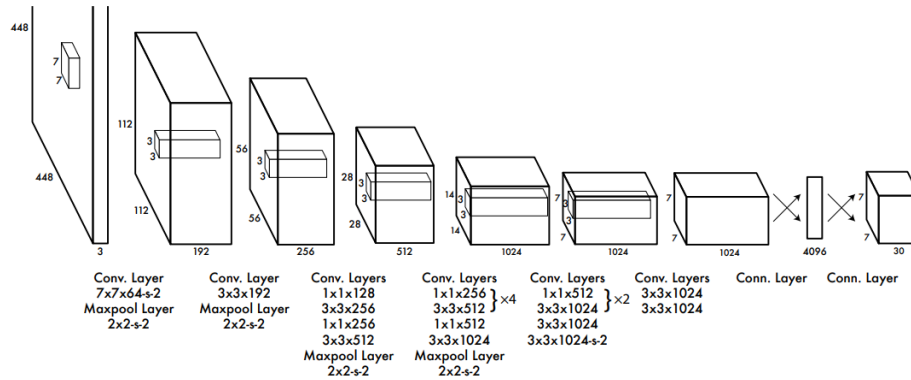


Figure 3: The Architecture. Our detection network has 24 convolutional layers followed by 2 fully connected layers. Alternating 1×1 convolutional layers reduce the features space from preceding layers. We pretrain the convolutional layers on the ImageNet classification task at half the resolution (224×224 input image) and then double the resolution for detection.

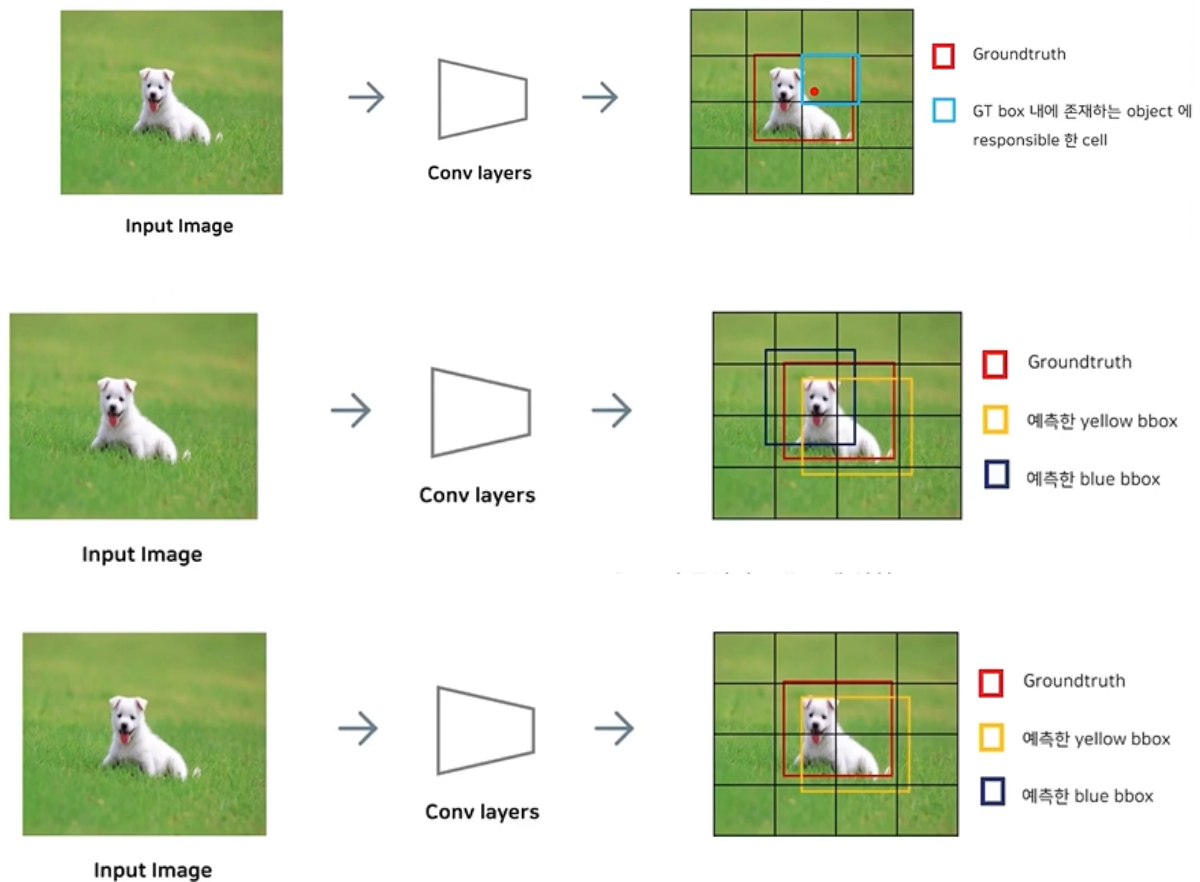
2.2 Training

- 앞 Convolution Layer 20개는 ImageNet 100-class 데이터셋으로 사전 학습 진행
- Ren et al. 연구를 참고하여, 추가로 4개의 Conv Layer와 2개의 Fully Connected Layer를 더함
- 입력 해상도를 224×224 에서 448×448 로 증가시켜 더 정교한 시각 정보를 활용
- 최종 레이어는 선형 활성화 함수(linear activation) => 출력값이 직접적인 수치로 해석되어야 하기 때문
- 나머지 레이어는 Leaky ReLU를 사용

Loss Function

- YOLO는 학습 과정에서 모든 bounding box를 활용하지 X
- GT box 의 중심에 위치한 grid cell만이 객체를 가짐
- 해당 responsible한 grid cell에서 IOU가 가장 높은 bbox 1개만 사용하여 학습 진행
pred,truth
- ex) 1 =1 로 표시

7yellow,object



- 합계 제곱 오차 (Sum-Squared Error) 사용
- bounding box 좌표 & 크기 loss
 - $1_{ij,object} = 1$ 일 때만 (객체가 있는 경우에만)
 - 바운딩 박스 크기에 따른 오차 차별을 없애기 위해 루트를 씌움
- bounding box confidence loss
 - $1 = 1$ or 0 일 때 둘 다 계산 (객체가 없는 경우에도)
- $ij,object$
- 클래스 확률 loss
- 이미지 대부분의 그리드 셀에는 객체가 없는 경우가 많음
- 따라서 모델은 객체가 없는 셀에서 신뢰도를 0으로 예측하는 작업에 더 집중하게 됨

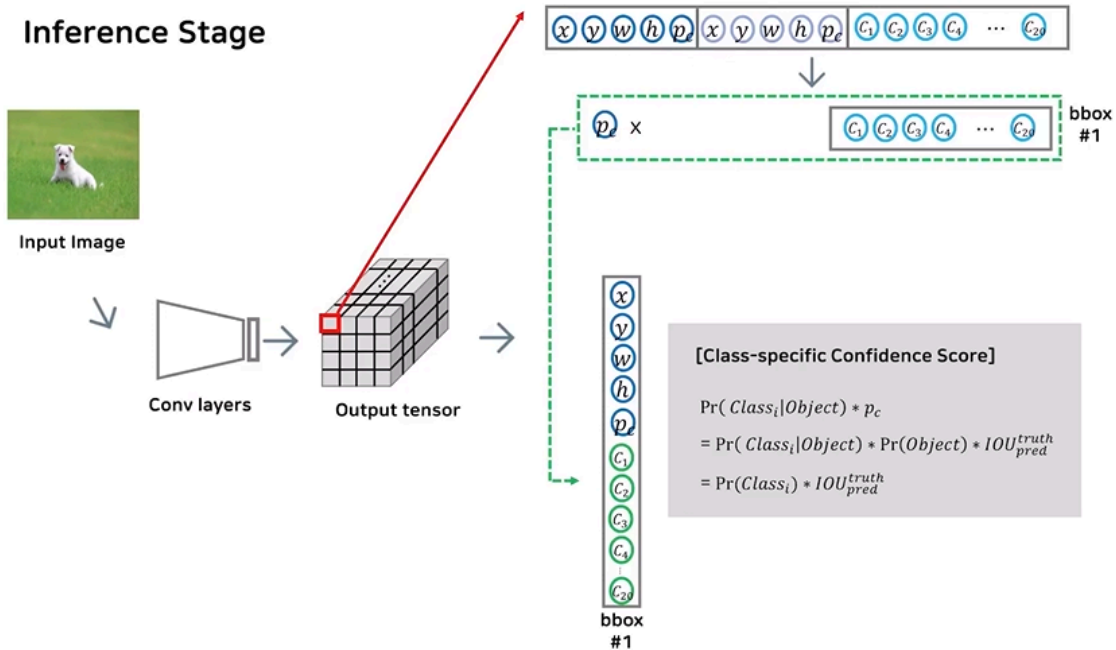
- 본 task에서는 bounding box 좌표 예측이 더 중요
- 이를 해결하기 위해 $\lambda = 5, \lambda = 0.5$ 설정

$$\begin{aligned}
& \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \\
& + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right] \\
& + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 \\
& + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 \\
& + \sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2
\end{aligned}$$

2.3 Inference

- Single network로 detection 가능함
- PASCAL VOC dataset 기준, 이미지 1개당 98개의 bounding box 생성 & class-specific confidence score
- object 당 bbox 개수가 많으므로 NMS 적용 필요

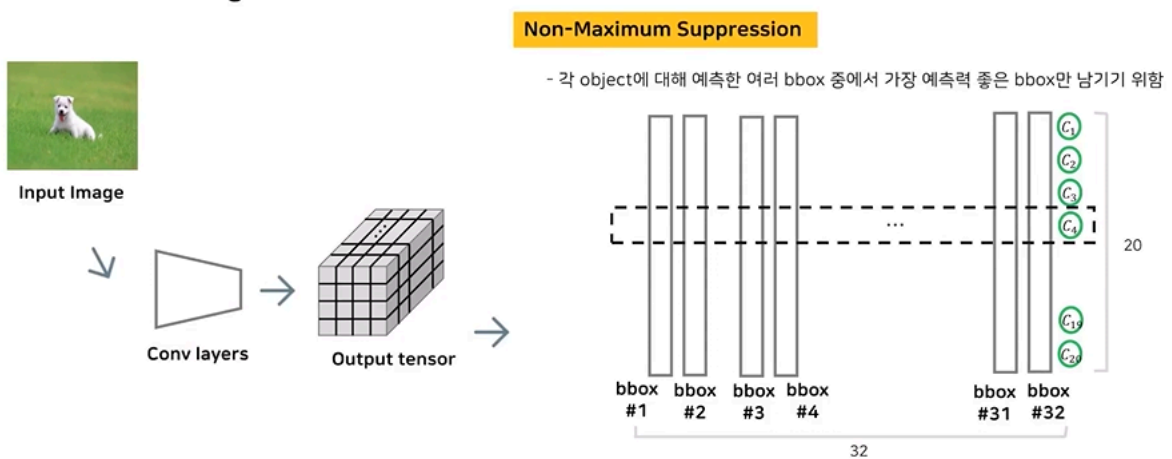
Inference Stage



22

- class마다 class-specific confidence score 정렬 후 NMS 적용
- 각 object에 대해 예측한 여러 bbox 중에서 가장 예측력이 좋은 bbox만 남기기 위함

Inference Stage



<https://www.youtube.com/watch?v=O78V3kwBRBk&t=1180s>

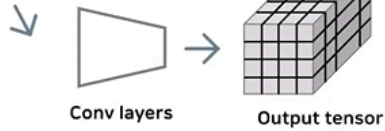
- 같은 class 속하는 object가 2개 이상인 경우
 - bbox #12와 #13의 IOU가 높으므로 NMS에 의해 제거됨
 - bbox #12와 #16의 IOU가 낮으므로 NSM에 의해 제거되지 않음

Inference Stage

2) 같은 class 속하는 Object 가 2개인 경우



Input Image



Non-Maximum Suppression

예시)

0.9	0.88	0.85	0.83	...	0.0	0.0
bbox #12	bbox #13	bbox #16	bbox #17		bbox #2	bbox #1

강아지



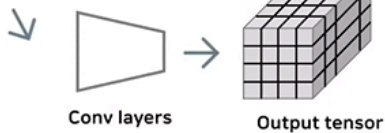
- Bbox#12 와 bbox#13의 IOU가 높으므로 NMS에 의해 제거됨.

Inference Stage

2) 같은 class 속하는 Object 가 2개인 경우



Input Image

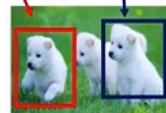


Non-Maximum Suppression

예시)

0.9	0.88	0.85	0.83	...	0.0	0.0
bbox #12	bbox #13	bbox #16	bbox #17		bbox #2	bbox #1

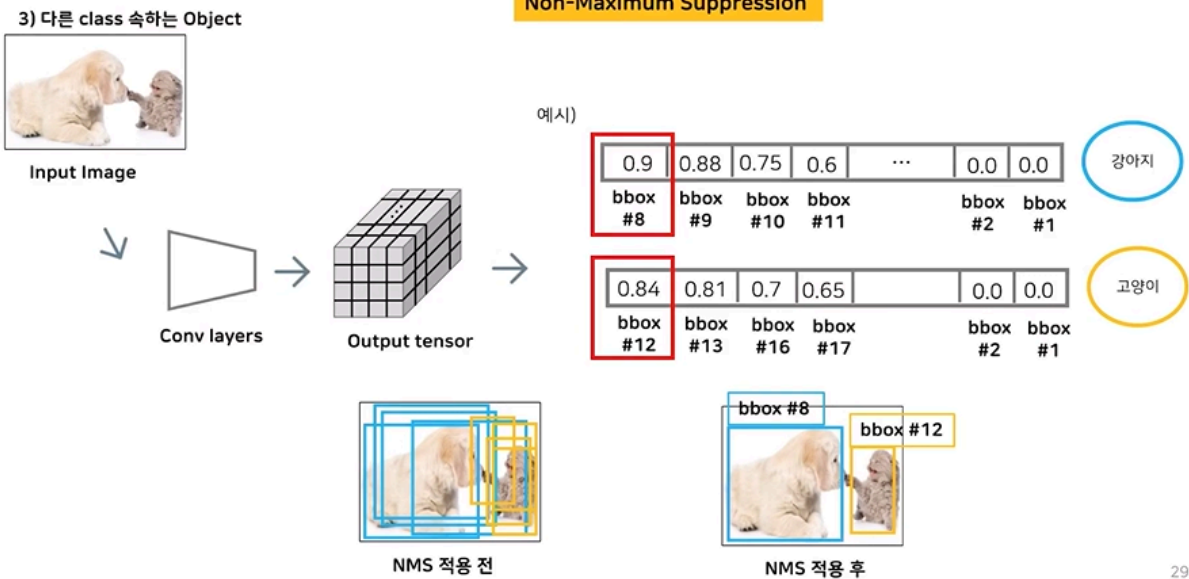
강아지



- Bbox#12 와 bbox#16의 IOU가 낮으므로 NMS에 의해 제거되지 않음.

- 다른 class에 속하는 object
 - class마다 class-specific confidence score 정렬 후 NMS 적용

Inference Stage



29

2.4 Limitations of YOLO

- YOLO는 각 그리드 셀이 두 개의 박스만 예측하고 한 클래스만 가질 수 있기 때문에 바운딩 박스 예측에 강한 공간적 제약을 가짐
- 이 제약 때문에 모델은 가까이 있는 여러 객체를 잘 예측하지 못함
 - 예를 들어, 새 떼처럼 작은 객체가 무리지어 있는 경우 문제가 발생
- 또한, YOLO는 데이터를 통해 바운딩 박스를 예측하는 법을 배우기 때문에 새로운 형태나 특이한 비율의 객체를 일반화하는 데 어려움을 겪음
- 그리고 네트워크 아키텍처가 입력 이미지를 여러 번 다운샘플링하기 때문에 바운딩 박스를 예측할 때 상대적으로 거친(coarse) 특징을 사용
- 마지막으로, YOLO는 감지 성능을 근사하는 손실 함수를 사용하여 학습하지만, 작은 바운딩 박스와 큰 바운딩 박스에서의 에러를 똑같이 취급
- 큰 박스에서의 작은 에러는 크게 문제가 되지 않지만, 작은 박스에서의 작은 에러는 IOU(교차 영역 비율)에 큰 영향을 미침

4. Experiments

4.1 Comparison to Other Real-Time Systems

- YOLO는 PASCAL VOC 2007에서 가장 빠르고 정확한 객체 탐지 모델
- Fast YOLO: 52.7% mAP로 이전 실시간 모델보다 2배 이상 정확

- VGG-16을 사용한 YOLO는 더 정확하지만 실시간 성능이 떨어짐
- Faster R-CNN: 높은 정확도를 제공하지만 속도(7fps)에서 YOLO(45fps)보다 훨씬 느림

Real-Time Detectors	Train	mAP	FPS
100Hz DPM [31]	2007	16.0	100
30Hz DPM [31]	2007	26.1	30
Fast YOLO	2007+2012	52.7	155
YOLO	2007+2012	63.4	45
Less Than Real-Time			
Fastest DPM [38]	2007	30.4	15
R-CNN Minus R [20]	2007	53.5	6
Fast R-CNN [14]	2007+2012	70.0	0.5
Faster R-CNN VGG-16[28]	2007+2012	73.2	7
Faster R-CNN ZF [28]	2007+2012	62.1	18
YOLO VGG-16	2007+2012	66.4	21

Table 1: Real-Time Systems on PASCAL VOC 2007. Comparing the performance and speed of fast detectors. Fast YOLO is the fastest detector on record for PASCAL VOC detection and is still twice as accurate as any other real-time detector. YOLO is 10 mAP more accurate than the fast version while still well above real-time in speed.

4.2 VOC 2007 Error Analysis

- YOLO의 주요 오류는 Localization Error(위치 오류): 객체의 위치를 정확히 잡지 못함
- Fast R-CNN은 Localization Error는 적지만, Background Error(배경 오류)가 많음
- YOLO는 Fast R-CNN에 비해 배경을 객체로 오탐지하는 확률이 3배 낮음

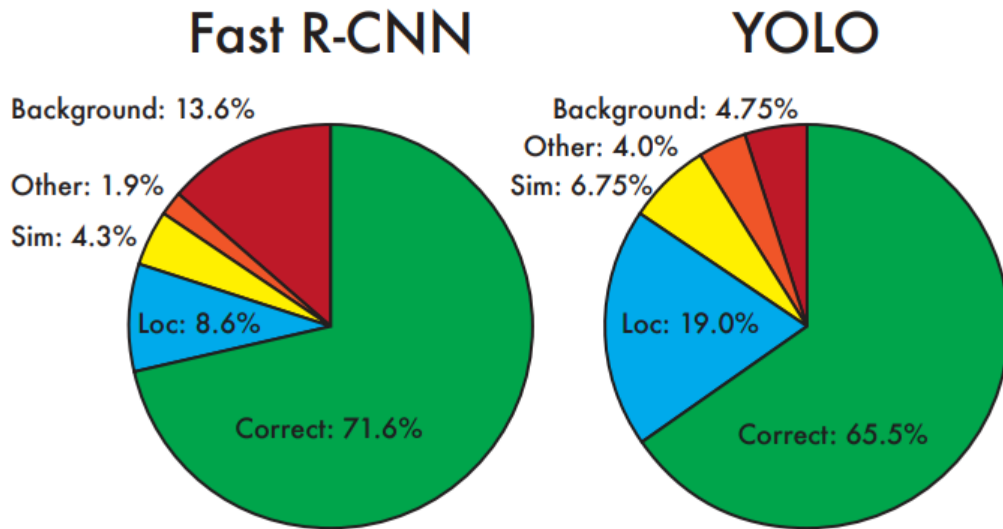


Figure 4: Error Analysis: Fast R-CNN vs. YOLO These charts show the percentage of localization and background errors in the top N detections for various categories (N = # objects in that category).

4.3 Combining Fast R-CNN and YOLO

- YOLO를 Fast R-CNN의 배경 오류 필터로 사용해 성능 향상
 - Fast R-CNN이 예측한 바운딩 박스가 진짜 객체인지 아닌지를 YOLO로 확인
- Fast R-CNN의 mAP가 71.8% → 75.0%로 3.2% 상승
- YOLO와 Fast R-CNN은 서로 다른 유형의 오류를 보완하여 성능을 극대화
- 하지만 속도는 Fast R-CNN의 느린 성능에 영향을 받음

	mAP	Combined	Gain
Fast R-CNN	71.8	-	-
Fast R-CNN (2007 data)	66.9	72.4	.6
Fast R-CNN (VGG-M)	59.2	72.4	.6
Fast R-CNN (CaffeNet)	57.1	72.1	.3
YOLO	63.4	75.0	3.2

Table 2: Model combination experiments on VOC 2007. We examine the effect of combining various models with the best version of Fast R-CNN. Other versions of Fast R-CNN provide only a small benefit while YOLO provides a significant performance boost.

4.4 VOC 2012 Results

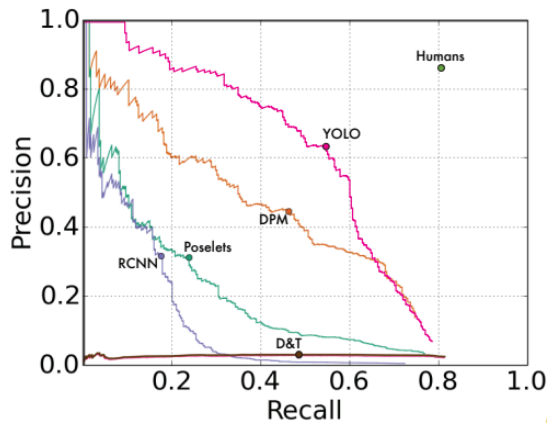
- YOLO: 57.9% mAP (최신 모델보다는 낮음)
- 작은 객체(예: 병, 양, TV)에서 약점이 있지만, 큰 객체(예: 고양이, 기차)에서는 더 높은 성능을 보임
- YOLO + Fast R-CNN 결합 모델은 VOC 2012 리더보드에서 상위권에 진입

VOC 2012 test	mAP	aero	bike	bird	boat	bottle	bus	car	cat	chair	cow	table	dog	horse	mbike	person	plant	sheep	sofa	train	tv
MR_CNN_MORE_DATA [11]	73.9	85.5	82.9	76.6	57.8	62.7	79.4	77.2	86.6	55.0	79.1	62.2	87.0	83.4	84.7	78.9	45.3	73.4	65.8	80.3	74.0
HyperNet_VGG	71.4	84.2	78.5	73.6	55.6	53.7	78.7	79.8	87.7	49.6	74.9	52.1	86.0	81.7	83.3	81.8	48.6	73.5	59.4	79.9	65.7
HyperNet_SP	71.3	84.1	78.3	73.3	55.5	53.6	78.6	79.6	87.5	49.5	74.9	52.1	85.6	81.6	83.2	81.6	48.4	73.2	59.3	79.7	65.6
Fast R-CNN + YOLO	70.7	83.4	78.5	73.5	55.8	43.4	79.1	73.1	89.4	49.4	75.5	57.0	87.5	80.9	81.0	74.7	41.8	71.5	68.5	82.1	67.2
MR_CNN_S_CNN [11]	70.7	85.0	79.6	71.5	55.3	57.7	76.0	73.9	84.6	50.5	74.3	61.7	85.5	79.9	81.7	76.4	41.0	69.0	61.2	77.7	72.1
Faster R-CNN [28]	70.4	84.9	79.8	74.3	53.9	49.8	77.5	75.9	88.5	45.6	77.1	55.3	86.9	81.7	80.9	79.6	40.1	72.6	60.9	81.2	61.5
DEEP_ENS_COCO	70.1	84.0	79.4	71.6	51.9	51.1	74.1	72.1	88.6	48.3	73.4	57.8	86.1	80.0	80.7	70.4	46.6	69.6	68.8	75.9	71.4
NoC [29]	68.8	82.8	79.0	71.6	52.3	53.7	74.1	69.0	84.9	46.9	74.3	53.1	85.0	81.3	79.5	72.2	38.9	72.4	59.5	76.7	68.1
Fast R-CNN [14]	68.4	82.3	78.4	70.8	52.3	38.7	77.8	71.6	89.3	44.2	73.0	55.0	87.5	80.5	80.8	72.0	35.1	68.3	65.7	80.4	64.2
UMICH_FGS_STRUCT	66.4	82.9	76.1	64.1	44.6	49.4	70.3	71.2	84.6	42.7	68.6	55.8	82.7	77.1	79.9	68.7	41.4	69.0	60.0	72.0	66.2
NUS_NIN_C2000 [7]	63.8	80.2	73.8	61.9	43.7	43.0	70.3	67.6	80.7	41.9	69.7	51.7	78.2	75.2	76.9	65.1	38.6	68.3	58.0	68.7	63.3
BabyLearning [7]	63.2	78.0	74.2	61.3	45.7	42.7	68.2	66.8	80.2	40.6	70.0	49.8	79.0	74.5	77.9	64.0	35.3	67.9	55.7	68.7	62.6
NUS_NIN	62.4	77.9	73.1	62.6	39.5	43.3	69.1	66.4	78.9	39.1	68.1	50.0	77.2	71.3	76.1	64.7	38.4	66.9	56.2	66.9	62.7
R-CNN VGG BB [13]	62.4	79.6	72.7	61.9	41.2	41.9	65.9	66.4	84.6	38.5	67.2	46.7	82.0	74.8	76.0	65.2	35.6	65.4	54.2	67.4	60.3
R-CNN VGG [13]	59.2	76.8	70.9	56.6	37.5	36.9	62.9	63.6	81.1	35.7	64.3	43.9	80.4	71.6	74.0	60.0	30.8	63.4	52.0	63.5	58.7
YOLO	57.9	77.0	67.2	57.7	38.3	22.7	68.3	55.9	81.4	36.2	60.8	48.5	77.2	72.3	71.3	63.5	28.9	52.2	54.8	73.9	50.8
Feature Edit [33]	56.3	74.6	69.1	54.4	39.1	33.1	65.2	62.7	69.7	30.8	56.0	44.6	70.0	64.4	71.1	60.2	33.3	61.3	46.4	61.7	57.8
R-CNN BB [13]	53.3	71.8	65.8	52.0	34.1	32.6	59.6	60.0	69.8	27.6	52.0	41.7	69.6	61.3	68.3	57.8	29.6	57.8	40.9	59.3	54.1
SDS [16]	50.7	69.7	58.4	48.5	28.3	28.8	61.3	57.5	70.8	24.1	50.7	35.9	64.9	59.1	65.8	57.1	26.0	58.8	38.6	58.9	50.7
R-CNN [13]	49.6	68.1	63.8	46.1	29.4	27.9	56.6	57.0	65.9	26.5	48.7	39.5	66.2	57.3	65.4	53.2	26.2	54.5	38.1	50.6	51.6

Table 3: PASCAL VOC 2012 Leaderboard. YOLO compared with the full comp4 (outside data allowed) public leaderboard as of November 6th, 2015. Mean average precision and per-class average precision are shown for a variety of detection methods. YOLO is the only real-time detector. Fast R-CNN + YOLO is the forth highest scoring method, with a 2.3% boost over Fast R-CNN.

4.5 Generalizability : Person Detection in Artwork

- Picasso Dataset, People-Art Dataset에서 YOLO는 기존 모델보다 새로운 도메인에 더 잘 일반화
- R-CNN은 자연 이미지에 특화되어 있어 성능이 급격히 저하됨
- YOLO는 객체의 크기, 모양, 관계를 잘 모델링하여 예술작품에서도 성능 유지



(a) Picasso Dataset precision-recall curves.

	VOC 2007 AP	Picasso		People-Art AP
		AP	Best F_1	
YOLO	59.2	53.3	0.590	45
R-CNN	54.2	10.4	0.226	26
DPM	43.2	37.8	0.458	32
Poselets [2]	36.5	17.8	0.271	
D&T [4]	-	1.9	0.051	

(b) Quantitative results on the VOC 2007, Picasso, and People-Art Datasets. The Picasso Dataset evaluates on both AP and best F_1 score.

Figure 5: Generalization results on Picasso and People-Art datasets.

5. Real-Time Detection In the Wild

- YOLO는 웹캠에 연결해 실시간 객체 탐지가 가능하며, 객체가 이동하고 외형이 바뀌어도 정확히 탐지
- 웹캠과 연결하면 실시간 추적 시스템처럼 작동
- YOLO의 빠르고 정확한 성능은 컴퓨터 비전 애플리케이션에 이상적



<https://devocean.sk.com/blog/techBoardDetail.do?ID=166976&boardType=techBlog>

6. Conclusion

- YOLO의 특징
 - YOLO는 객체 탐지를 위한 통합된 모델로, 전체 이미지를 한 번에 처리
 - 기존 분류 기반 접근법과 달리, 탐지 성능에 직접적으로 연관된 손실 함수로 훈련
 - 모델 전체를 통합적으로 학습하도록 설계
- 속도와 성능
 - Fast YOLO: 현재 문헌에서 가장 빠른 범용 객체 탐지기
 - YOLO: 실시간 객체 탐지에서 최첨단 성능을 제공하며, 속도와 정확도를 모두 갖추
- 일반화 능력
 - YOLO는 새로운 도메인(예: 예술 작품)에도 잘 일반화 가능
 - 빠르고 강력한 객체 탐지가 필요한 애플리케이션에 적합

Abstract

- NLP에서 사용된 Transformer Architecture가 많이 사용되지만 CV 분야에서 응용은 제한적으로 사용됨
 - 기존의 CV에서 사용된 것은 Attention 구조 + Convolutional Network 혹은 Conv network 란

▼ Self - Attention

- Long-range Dependencies
 - 전체 입력 시퀀스에서 시퀀스들 간의 관계를 학습하는 역할
 - ⇒ 이미지에서 멀리 떨어진 픽셀이나 패치 간의 관계를 효과적으로 포착할 수 있다
- 주어진 문장에서 특정 시퀀스가 다른 시퀀스와 어떤 관계를 맺고 있는지를 파악
 - 출력 생성 시 더 관련성 높은 단어에 집중

1. Introduction

1. 배경

- NLP 분야에서의 Attention의 영향
 - Self-attention기반 아키텍처는 NLP분야의 태스크에서 보편적으로 쓰이는 모델 (e.g. Transformer)
 - 일반적으로 Large text corpus에 pre-training → smaller task-specific dataset에 전이학습을 시키는 접근법이 널리 사용
 - Transformer-based 모델의 장점
 1. 연산이 효율적이다(computational efficiency).
 2. 확장성이 좋다(scalability). 특히 input sequence의 길이에 구애받지 않는다.
- CV 의 구조 및 한계
 - 하지만, computer vision 분야에서는, 여전히 convolutional-based architecture가 지배적

- 물론 NLP 분야에서의 성공을 참고 삼아 CNN기반 아키텍처를 Self-attention과 결합하거나, CNN을 아예 Attention으로 대체하는 연구들도 존재
((2018)Wang et al. / (2019)Ramachandran et al.)
- ➔ 이 연구들은 이론적으로 효율적이긴 했지만, GPU(TPU)와 같은 하드웨어 가속기에 보편적으로 적용하지 못하는 상태
- ⇒ 이런 저런 이유들로 여전히 Res-net기반 아키텍처들이 이미지 분야에서의 SOTA모델인 상황

2. 연구

- 저자들은 NLP 분야에서 트랜스포머의 **scaling successes**에 특히 영감을 받아 image에 트랜스포머를 **직접적으로 적용할 수 있는** 연구를 진행
- 간략한 ViT 방법
 1. 이미지를 image pathces로 쪼갬다.
 2. 이 patches들을 linear하게 임베딩해 시퀀스를 생성한다.
 3. 이렇게 생성한 시퀀스를 트랜스포머의 input으로 투입한다.
- ⇒ 즉, ViT에서 하나의 이미지 패치는 하나의 token으로 작동

3. Inductive Bias

▼ inductive bias란?

모든 모델이 **지금까지 만나보지 못했던 상황에서 정확히 예측하기 위해 사용하는 추가적인 가정** → 사전지식과 동일한 역할

• CNN : **Locality**와 **translation equivariance**

◦ Locality

이미지에서 중요한 특징은 일반적으로 작은 영역(지역적 정보)에 존재하고, 이 특징은 이미지의 다른 부분으로 이동해도 그 의미를 유지한다는 가정

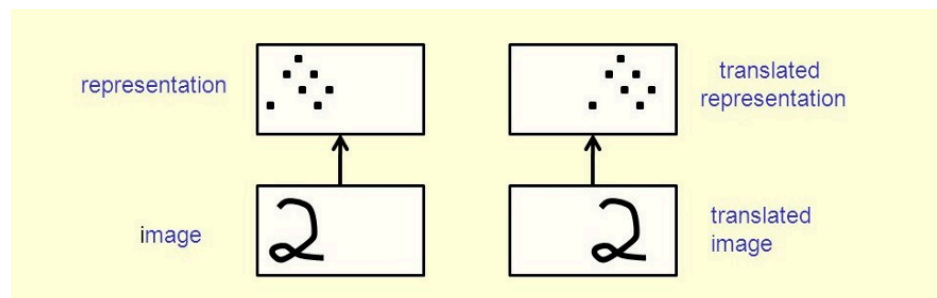
⇒ Convolutional filter가 작은 receptive field를 통해 지역적인 패턴을 추출하는 방식으로 이를 반영

◦ Translation Equivariance

객체가 이미지 내에서 이동해도 객체의 특징을 감지하는 메커니즘은 동일하게 적용될 수 있다

⇒ Convolutional layer는 동일한 필터를 이미지의 모든 공간 위치에 적용하는 방식으로 이를 반영

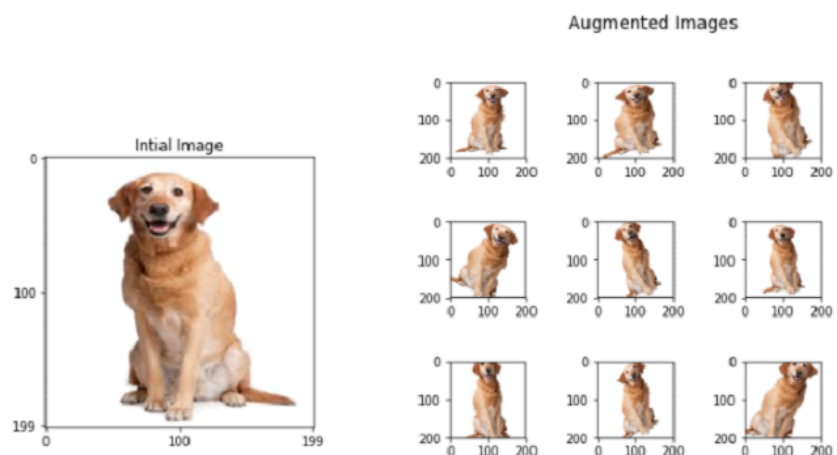
- 간단히 말하면, CNN은 convolution 과정을 수행할 때 **가중치가 공유되는 filter**를 이용해 새로운 이미지를 생성하기 때문에, input image 패턴만 같다면 output representation 또한 같은 패턴을 생성
- 즉, **object를 단순히 translation(이동)** 시킨다면 아래와 같이 유사한 output representation을 생성합니다.



- 위의 예시에서 [pooling→softmax]를 통한 분류를 수행한다면, representation의 위치는 변화할지 일정 결과적으로 같은 class를 반환한다는 것을 쉽게 유추
- 반대로 단순 translation(이동)이 아닌 **rotation(회전)**이나 **scale(크기)**이 변환한다면 결과 또한 달라짐.

→ data augmentation

- 그렇기 때문에 data augmentation을 적용할 때 단지 Translation만 적용한다면 사실상 의미가 없음



- RNN : **Sequentiality**를 사용해 패턴을 예측

⇒ CNN은 local한 영역에서 공간정보를 뽑아내고, RNN 역시 시계열 관점에서 시간에 따른 Sequential한 정보를 뽑아내 예측에 사용

- Transformer는 이런 가정이 없어서, 많은 데이터로부터 충분한 학습을 하지 않으면 성능이 좋지 못함
- 반대로 말하면, CNN과 RNN 둘 다 global한 정보를 활용하기는 힘들기 때문에 한계 역시 명확

Component	Entities	Relations	Rel. inductive bias	Invariance
Fully connected	Units	All-to-all	Weak	-
Convolutional	Grid elements	Local	Locality	Spatial translation
Recurrent	Timesteps	Sequential	Sequentiality	Time translation
Graph network	Nodes	Edges	Arbitrary	Node, edge permutations

Table 1: Various relational inductive biases in standard deep learning components. See also Section 2.

- **ImageNet**과 같은 mid-sized dataset에 ViT를 학습시킬 때에는, 강한 규제가 없다면, 비슷한 사이즈를 갖는 ResNet모델에 비해 정확도가 몇 퍼센트 낮은 무난한 결과를 보임
 - 성능 자체는 CNN기반 ResNet보다 낮음
 - Transformer는 inductive biases가 낮음
 - CNN은 inductive biases가 높음
- ViT는 이런 CNN에 내재된 inductive bias가 존재하지 않기 때문에, 데이터가 적을 때 성능 자체는 낮을 수 있지만, **larger datasets(14M-300M개의 이미지)**에 학습시킨다면, 이런 inductive bias의 효과를 추월할 수 있음
- 즉, **ImageNet-21k나 JFT-300M dataset**과 같은 큰 데이터셋에 사전학습 시킨 다음, 각각의 태스크에 전이학습을 시켰을 경우 ViT는 거의 sota성능

지금까지 많이 나온 scale, scalability 등은 트랜스포머 모델의 태스크 확장성, large dataset 활용능력 등을 뜻하는 부분

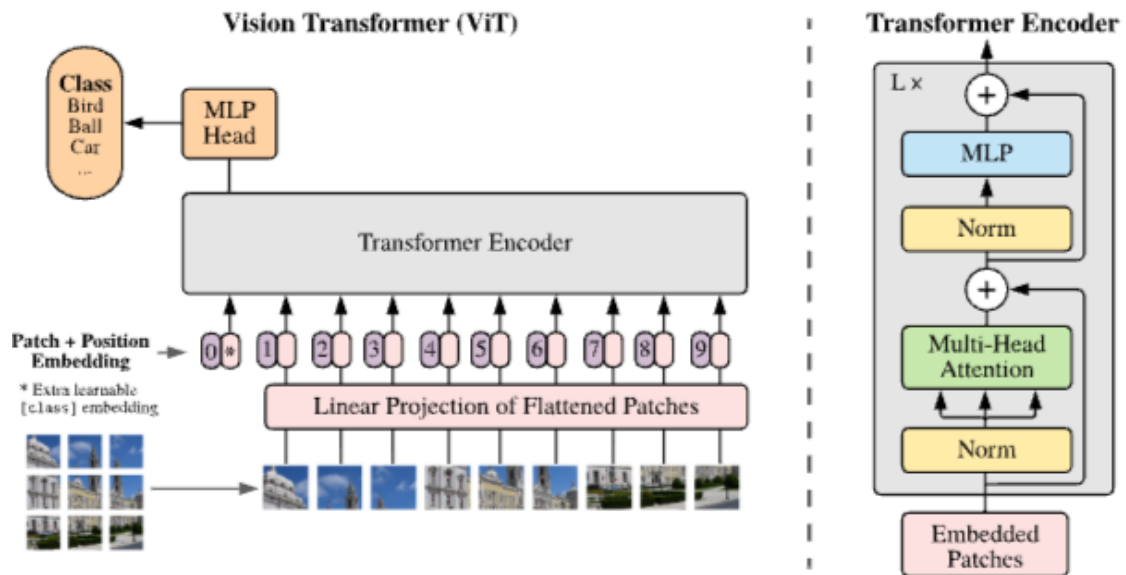
⇒ 적은 데이터셋에서는 inductive bias가 효과적으로 작용할 수 있지만, 대규모 데이터셋에서는 오히려 inductive bias로 인해 제약이 발생할 가능성에 주목.

▼ 왜 inductive bias가 데이터셋이 적을 때, 효과적으로 작용할 수 있는 것인지?

- 적은 데이터 셋 = 모델이 학습할 수 있는 정보의 양이 제한적이라 모델이 특정 구조나 패턴을 발견하는 게 어려움
→ 이때 inductive bias가 학습 가이드의 역할을 수행해줌.
 - sample efficiency 증대
 - CNN은 특정 특징(예: 눈의 가장자리)을 한 위치에서 학습하면, 그 특징이 이미지의 다른 위치에 나타나더라도 다시 학습할 필요 없이 재활용할 수 있음
 - 데이터가 부족한 상황에서는 이러한 "재활용" 능력이 모델이 빠르게 일반화 가능한 특징을 학습하게 하여 과적합(overfitting)을 줄이고 성능을 향상
- ⇒ 데이터가 적어도 모델이 "이미지란 이런 방식으로 생겼을 것이다"라는 강한 가정을 가지고 학습을 시작
- 탐색 공간 축소 (Reduced Search Space)
 - 모델이 탐색해야 할 가중치 공간(weight space)을 효과적으로 줄여줌
 - CNN은 필터가 지역적으로 연결되고 공유되는 구조를 가지므로, 학습해야 할 매개변수의 수가 줄어들고, 따라서 학습이 더 안정적이고 효율적
 - 제한된 탐색공간으로 의미있는 특징을 찾는 것에 더 큰 도움을 줌

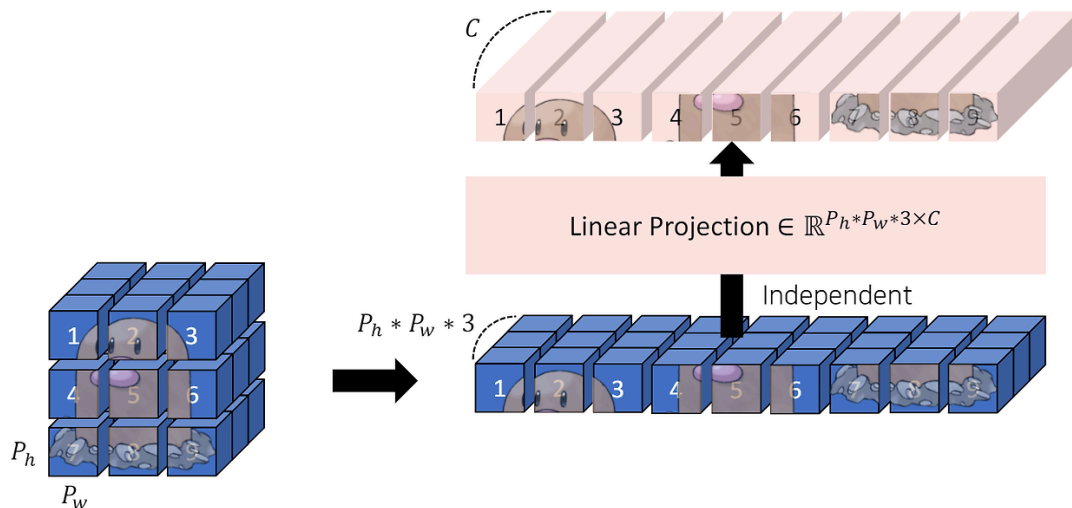
2. Related Work

- NLP에서의 Transformer
 - introduction에 써있듯 NLP에서는 large corpus pre-training → downstream task에 전이학습 시켜서 사용
 - BERT : denoising self-supervised pre-training task 사용
 - GPT 계열 : language modeling을 pre-training task로 사용



3. Method

3.1 Vision Transformer



[입력 토큰 임베딩]

- Transformer는 1D sequence를 입력으로 받음.
- 이미지 $x_p \in \mathbb{R}^{H \times W \times C} \rightarrow \text{flatten} \rightarrow x_p \in \mathbb{R}^{N \times (P^2 \cdot C)}$
 - H, W : 해상도

- C : 채널 수
- P : patch 크기 (실험: 14, 16, 32 등)
- $N = HW/P^2$: patch 개수
- Linear Projection E를 통해 각 patch를 D차원 latent vector로 변환.

[CLS 토큰]

- BERT와 동일하게 [CLS] token을 추가 → 이미지 전체 표현을 담당.
- Encoder 최종 출력의 첫 번째 토큰을 classification head로 전달.
 - Pre-training: hidden layer 1개 포함된 MLP head
 - Fine-tuning: 단일 linear head
- 즉, classification은 encoder의 첫 번째 벡터 y를 MLP head에 통과시켜 수행.

[Positional Embedding]

- Patch로 쪼개면 위치 정보가 사라지므로 positional embedding 추가.
- 이미지라 해도 **1D positional embedding**이 사용됨. (2D는 성능 향상도 없고 효율도 떨어짐)
- Fine-tuning 시 입력 해상도가 다를 경우 → pre-trained positional embedding을 **2D interpolation**으로 보간 후 사용.

[Encoders]

1. Multi-Head Self Attention Layer (MSP)

- 모든 attention outputs을 올바른 차원에 linearly하게 연결한다.
- 많은 attention heads는 이미지의 local 및 global dependencies를 학습하는데 도움 줌

2. Multi-Layer Perceptrons (MLP) Layer

- Gaussian Error Linear Unit(GELU)가 있는 두 개의 레이어가 포함.

3. Layer Norm (LN)

- Train image간의 새로운 dependencies를 포함하지 않기 때문에 각 block 앞에 추가
- Training time과 overall performance를 향상시키는데 도움.

4. Residual connections

- component가 non-linear activation function을 거치지 않고 네트워크를 통해 직접 흐를 수 있도록 block 뒤에 포함.

Inductive bias

- CNN: **locality 가정** (주변 pixel 간 연관) + **translation equivariance** (이동 불변성)
- RNN: **sequentiality 가정** (시간적 순서)
- SVM: **margin 최대화 가정**
- 이로 인해 global 관계 학습은 제한적.

→ 반면 ViT는 이러한 inductive bias가 거의 없음.

- 장점: 더 유연하게 global 관계를 학습 가능.
- 단점: bias가 없기 때문에 더 많은 데이터셋 필요.
- ViT는 self-attention을 통해 global context를 학습하지만, spatial 관계는 처음부터 다시 학습해야 함.

Hybrid Architecture

- Patch embedding 대신 CNN backbone feature map 사용 가능.
- CNN feature → flatten → embedding projection E
- ResNet feature map을 transformer 입력으로 사용 → CNN의 local feature 추출 + ViT의 global context 결합.
- 효율성과 성능을 동시에 추구하는 구조.

3.2 Fine Tuning and higher resolution

- Fine-tuning 시 pre-training보다 더 높은 해상도 입력 사용 가능.
- Patch size 동일 시 → sequence length NNN 증가.
- 이 경우 pre-trained positional embedding 크기 불일치 → **2D interpolation**으로 보간 후 재사용

▼ higher resolution

- ViT는 이미지를 **패치 단위**로 나누고, 각 패치에 위치 정보를 담기 위해 **Position Embedding**을 더합니다.
- 그런데 **Fine-Tuning** 단계에서 **더 높은 해상도**의 이미지를 넣으면, ****패치 개수 (N)****가 달라집니다.
 - 예: Pre-training에서는 224×224 해상도 → 16×16 패치 → 총 196개 패치
 - Fine-tuning에서는 384×384 해상도 → 16×16 패치 → 총 576개 패치
- 이렇게 되면 ****원래 학습된 Position Embedding의 크기(196개)****와 ****새로운 입력 크기(576개)****가 서로 안 맞습니다.

2. 해결 방법: 2D Interpolation

- 원래 학습된 Position Embedding은 **224×224 기준의 14×14 격자** 위에 정의되어 있습니다.
- Fine-Tuning 시 필요한 건 **384×384 기준의 24×24 격자**입니다.
- 그래서 기존 14×14 격자에 있던 값을 ****보간(interpolation)****해서 24×24 격자에 맞게 “늘려서” 씁니다.
- 즉, 기존 Position Embedding을 그대로 버리는 게 아니라, **2차원 이미지처럼 취급해서 사이사이에 값을 채워 넣는 방식**입니다.

4. Experiment

4.1 Setup

Model	Layers	Hidden size D	MLP size	Heads	Params
ViT-Base	12	768	3072	12	86M
ViT-Large	24	1024	4096	16	307M
ViT-Huge	32	1280	5120	16	632M

- Representation Learning & Benchmarks
 - ResNet, ViT, Hybrid 모델의 representation learning 성능을 평가.
 - 데이터셋 크기를 다양하게 변화시키며 pre-training 후 benchmark 수행.
 - 결과: ViT가 계산 비용이 더 낮음에도 SOTA 달성.
 - Self-supervision 실험도 진행 → ViT가 미래에 유망한 self-supervised 모델이 될 가능성 확인.

- Model Variants
 - ViT-Base, ViT-Large, ViT-Huge → 모델 크기에 따라 파라미터 수와 성능 차이.
 - 표기: 예) ViT-L/16 = Large 모델 + patch size 16×16.
 - Patch size ↓ → sequence length ↑ → 계산량 ↑.
 - Baseline CNN: ResNet(BiT) 사용 (BN 대신 GN, standardized convolution 적용).
- Hybrid Model Experiments
 - ResNet feature map을 ViT 입력으로 사용.
 - 실험 옵션:
 1. ResNet50 4단계 출력 사용
 2. 4단계를 제거하고 3단계 확장 → 시퀀스 길이 4배 증가
 - 목적: CNN의 local feature와 ViT의 global context 결합 → 더 강력한 모델.
- Training & Fine-Tuning Details
 - Optimizer: Adam ($\beta_1=0.9$, $\beta_2=0.999$)
 - Batch size: 4096 (pre-train), 512 (fine-tune)
 - Regularization: weight decay=0.1, dropout, label smoothing
 - Learning rate schedule: warmup + decay
 - Fine-tuning: SGD + momentum
- Metrics
 - **Fine-tuning Accuracy:** 특정 데이터셋에서의 성능
 - **Few-shot Accuracy:** 제한된 데이터로 평가. (고정된 표현을 $\{1, -1\}^K$ target vector로 회귀)

4.2 Comparison to SOTA

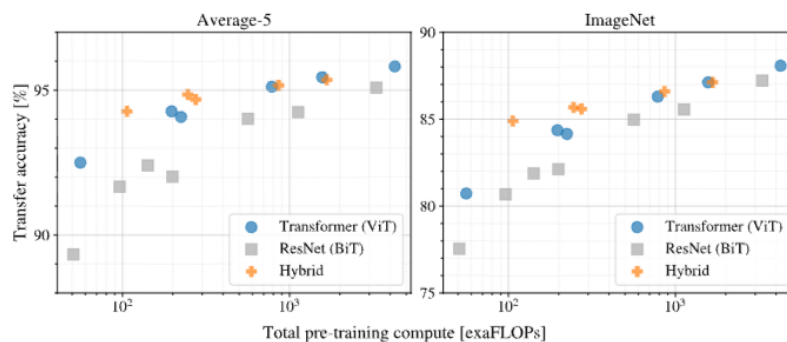
	Ours-JFT (ViT-H/14)	Ours-JFT (ViT-L/16)	Ours-I21k (ViT-L/16)	BiT-L (ResNet152x4)	Noisy Student (EfficientNet-L2)
ImageNet	88.55 ± 0.04	87.76 ± 0.03	85.30 ± 0.02	87.54 ± 0.02	88.4/88.5*
ImageNet Real	90.72 ± 0.05	90.54 ± 0.03	88.62 ± 0.05	90.54	90.55
CIFAR-10	99.50 ± 0.06	99.42 ± 0.03	99.15 ± 0.03	99.37 ± 0.06	—
CIFAR-100	94.55 ± 0.04	93.90 ± 0.05	93.25 ± 0.05	93.51 ± 0.08	—
Oxford-IIIT Pets	97.56 ± 0.03	97.32 ± 0.11	94.67 ± 0.15	96.62 ± 0.23	—
Oxford Flowers-102	99.68 ± 0.02	99.74 ± 0.00	99.61 ± 0.02	99.63 ± 0.03	—
VTAB (19 tasks)	77.63 ± 0.23	76.28 ± 0.46	72.72 ± 0.21	76.29 ± 1.70	—
TPUV3-core-days	2.5k	0.68k	0.23k	9.9k	12.3k

- ViT-L/16 (JFT-300M pre-train) > BiT-L (ResNet 기반 SOTA)
- ViT-H/14는 성능이 더 뛰어남.
- 심지어 ImageNet-21k(14M)만으로도 좋은 성능 확보.
- Pre-training cost도 ResNet(BiT) 대비 저렴.

4.3 Pre-training Data Requirements

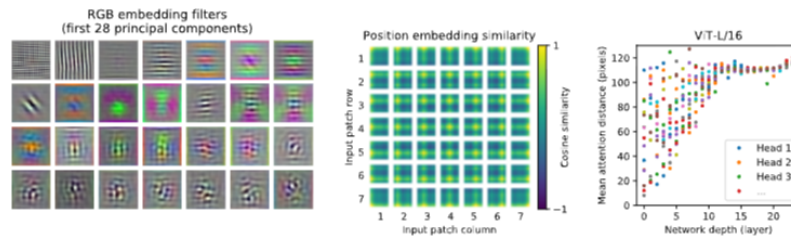
- 작은 데이터셋 (ImageNet, ImageNet-21k)에서는 ViT-Large가 underperform → inductive bias 부족 때문.
- JFT-300M 같은 대규모 데이터셋에서는 ViT의 장점이 극대화됨.
- 결론: ViT는 데이터 크기에 강하게 의존.

4.4 Scaling Study



- 다양한 크기의 ResNet, ViT, Hybrid 모델을 비교.
- 작은 모델: Hybrid > ViT
- 큰 모델: ViT > BiT
- ViT 성능은 아직 saturation되지 않음 → scaling으로 더 개선 여지.

4.5 Inspecting ViT



- 첫 번째 layer: patch projection matrix → conv filter와 유사.
- 두 번째 layer: learned position embedding → 가까운 patch끼리 높은 유사도 학습.
- Attention head 분석:
 - shallow layer: localization 효과 (CNN과 유사)
 - deeper layer: receptive field 확장 → global 정보 반영
 - Hybrid 모델: local attention 효과가 더 강함.

4.6 Self-Supervision

- BERT처럼 masking 기반 self-supervision 시도.
- ViT-B/16 → 79.9% 정확도 (supervised보다 낮지만 가능성 확인).

5. Conclusion

- ViT는 **CNN의 image-specific inductive bias 없이** self-attention만으로 작동.
- 대규모 데이터셋(JFT-300M)에서 뛰어난 성능, 비용 효율성도 ResNet(BiT)보다 우수.
- 향후 과제: Detection, Segmentation, Self-supervised learning 확장.
- ViT는 scaling law에 따라 더 큰 모델과 데이터셋에서 추가 성능 향상이 가능 → 발전 가능성이 큼.